Internship Report on

Drowsiness Car Driving

Submitted by

Abhishek Kumar

For the Award of

COMPLETION CERTIFICATE

In

Six-Week Summer Internship on Artificial Intelligence

Under the Supervision of

Prof. **Sanjay Kumar Soni**

HOD, ECED, MMMUT, GORAKHPUR

(Duration: May 13, 2025 to June 20, 2025)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

MADAN MOHAN MALAVIYA UNIVERSITY OF TECHNOLOGY

GORAKHPUR (U.P.) INDIA

Declaration

I, **Abhishek Kumar**, a student of the **Department of Data Science**, Institute of Engineering and Technology, hereby declare that the project report titled.
"Drowsiness Car Driving," submitted in partial fulfillment of the requirements for the Bachelor of Technology, is my original work.

I confirm that this report has not been submitted to any other university or institution for the award of any degree, diploma, or fellowship. The work presented in this report is based on my original work carried out under the guidance of **Prof. S.K. Soni** from **MADAN MOHAN MALAVIYA UNIVERSITY OF TECHNOLOGY, GORAKHPUR (U.P).**

I have duly acknowledged all the sources of information used in this report. Any data, analysis, or conclusions drawn in this report are solely my responsibility.

Signature of the Stu
Abhishek Kumar
Department of Data
Science

Indian Institute of
Technology Madras
(Chennai) Tamil Nadu

ACKNOWLEDGEMENT

I want to express my sincere gratitude to all those who have contributed to the successful completion of this project titled **“Drowsiness Car Driving”.**

First and foremost, I am deeply thankful to [Ph.D Shivam Kumar Singh], my project guide, for their valuable guidance, constant support, and constructive feedback throughout this project. Their expertise and encouragement were instrumental in shaping the direction and execution of the work.

This project provided me with the opportunity to learn about various aspects, including how health insurance works, the factors that affect its cost, and how data can be utilized to make predictions using machine learning.

I was amazed to see how technology is helping in the driving field by making this project ‘Be Safe Drive Safe’

ABSTRACT

Drowsiness during driving is one of the leading causes of road accidents worldwide. This project, titled **"Drowsiness Car Driving Detection System"**, aims to reduce such incidents by alerting drivers when signs of sleep or fatigue are detected. The system utilizes a webcam to continuously monitor the driver’s facial features in real time. Using the **MobileNet** deep learning model integrated with **TensorFlow**, **OpenCV (cv2)**, and **OS** libraries, the application detects eye closure and facial movements that indicate drowsiness.
If prolonged eye closure or signs of sleep are identified, an alarm is triggered to wake the driver and prevent potential accidents. The core objective is to create a low-cost, real-time solution that enhances road safety by providing early warnings of driver fatigue. Through this implementation, the system effectively contributes to reducing road accidents caused by inattentiveness and drowsiness behind the wheel.
This project has the following goals:

- Captures live video from the webcam
- Monitors eye movement and facial behavior

- Detects signs of drowsiness, such as prolonged eye closure

- Triggers an alarm if drowsiness is detected

INTERNSHIP OBJECTIVE

To apply and expand my technical skills in computer vision, embedded systems, and IoT by working on real-world applications such as driver assistance systems, while gaining hands-on experience with tools like OpenCV, TensorFlow, and microcontrollers in a professional environment.

- Gain hands-on experience in real-time hardware/software integration.
- Apply theoretical knowledge to solve practical engineering problems.
- Enhance skills in embedded systems, IoT, and computer vision technologies.
- Learn professional development workflows and project management practices.
- Work on real-world applications, especially in safety, automation, or smart systems.
- Explore industry tools such as TensorFlow, OpenCV, Arduino, NodeMCU, etc.
- Collaborate with experienced professionals and understand team dynamics.
- Understand how R&D or product development works in a professional setup.
- Improve debugging, testing, and documentation skills in live projects.
- Build a foundation for future career roles in AI/ML, IoT, and automation.

TABLE OF CONTENTS

INTRODUCTION

1.1 The **Drowsiness Car Driving Detection System** is a computer vision-based safety application that uses machine learning models to detect signs of fatigue or drowsiness in drivers. By analyzing facial landmarks such as eye closure and blinking patterns in real-time video captured from a webcam, the system alerts drivers when they show symptoms of sleepiness, thereby helping to prevent road accidents. This project is developed using deep learning techniques and lightweight models like MobileNet, along with libraries such as TensorFlow and OpenCV.

1.2 Definition of ML-

Machine Learning (ML) is a branch of artificial intelligence (AI) that focuses on the development of algorithms and statistical models that enable computers to learn from and make predictions or decisions based on data.

Definition of Computer Vision and Deep Learning

Computer Vision is a field of artificial intelligence that trains computers to interpret and understand the visual world. Using digital images from cameras and deep learning models, machines can accurately identify and classify objects, and then react to what they "see."

Deep Learning is a subset of machine learning that uses neural networks with many layers (hence "deep") to model complex patterns in data, especially useful for image, audio, and video processing.

1.3 Working of ML-

The system works through a structured pipeline involving real-time image capture, model inference, and alert generation. The major steps are:

1. **Data Acquisition**
2. Capture real-time video using a webcam placed in front of the driver.
3. The system continuously monitors facial features, especially the eyes and head movement.
4. **Model Selection**

   A pre-trained **MobileNet** model (or a similar CNN model) is used for face and eye detection due to its lightweight and efficient performance on real-time data.

5. **Feature Extraction and Analysis**
6. Using OpenCV and facial landmark detection, key features like Eye Aspect Ratio (EAR) are calculated to determine eye openness.
7. If eyes remain closed beyond a threshold (indicating sleep), it is flagged as drowsiness.

8. **Alert Mechanism**

   If drowsiness is detected, a buzzer or alarm is triggered to alert the driver immediately.

9. **Model Optimization**

   A performance is improved using real-world test data, hyperparameter tuning, and techniques like frame skipping, threshold calibration, and noise filtering.

10. Model Optimization- Tune parameters or use advanced techniques to improve accuracy (like cross-validation, feature selection, etc.).

The aim of this project is to develop a **Drowsiness Detection System for drivers** using machine learning and computer vision techniques. The system monitors the driver's facial Features—especially eye movement and blinking rate—through a webcam to detect signs of fatigue or sleepiness in real time. Upon detecting drowsiness, the system activates an alarm to alert the driver and prevent potential road accidents.

This project aims to enhance road safety and assist in reducing accident rates caused by drowsy driving by using lightweight deep learning models like **MobileNet**, along with libraries such as **TensorFlow** and **OpenCV**.

This project also aims to:

- 🧠 **Understand the role of eye and facial movement in detecting drowsiness.**
- 🎯 **Use computer vision and machine learning techniques to build a real-time alert system.**
- 🧪 **Collect, process, and analyze real-time video data using webcam feeds.**
- 🛠️ **Gain practical knowledge of deploying deep learning models for safety applications.**
- 🚗 **Demonstrate how AI-based systems can help make driving safer and reduce human error.**

Components Used

3.1 Software Components :

 Python Programming Language – Used to write the entire codebase, including image processing, model inference, and alert mechanisms.
 TensorFlow / Keras – For loading and using the pre-trained MobileNet model for face and eye detection.
 OpenCV (cv2) – For capturing webcam video, detecting facial landmarks, and image preprocessing.
 NumPy – For numerical operations and handling arrays.
 Imutils / dlib (optional) – For additional image processing and facial landmark detection.
 OS – To interact with the system and trigger alerts (e.g., buzzers or sound alarms).

- **Dataset :**

This project may use **pre-trained models like MobileNet** or **face landmark datasets** for training or fine-tuning. Alternatively, real-time webcam data is used directly for live testing without the need for a separate labeled dataset.

3.2 Hardware Components :

Here are the main hardware components used for real-time drowsiness detection:

- **Webcam** – Captures live video of the driver's face; essential for monitoring eye and facial behavior.
- **Computer or Laptop** – Used to run the Python-based detection system with model processing.
- **Processor (CPU)** – A modern dual-core or quad-core processor (Intel i5/Ryzen 5 or above) for real-time video processing.
- **Memory (RAM)** – Minimum 4–8 GB RAM for smooth operation of the software and webcam stream.
- **Speaker/Buzzer (optional)** – Used to alert the driver when drowsiness is detected.
- **Microcontroller (optional)** – Like NodeMCU or Arduino if you extend the system to hardware-based alerts.

4. WORKING PROCEDURE

The working procedure of the **Drowsiness Car Driving Detection System** using computer vision and a deep learning-based MobileNet model is as follows:

1. Real-Time Video Capture

The project begins with capturing live video feed using a webcam mounted near the driver. The camera focuses on the driver's face throughout the journey to monitor visual cues such as:

- Eye movement
- Blink frequency
- Duration of eye closure
- Head posture (optional)

2. Face and Eye Detection

2. Face and Eye Detection

Using **OpenCV** and a **pre-trained MobileNet model**, the system detects and isolates facial landmarks in each video frame. Important steps include:

- Detecting the face region from the frame
- Extracting eye regions (left and right eye)
- Calculating the Eye Aspect Ratio (EAR) or detecting closed eyes directly using a CNN

3. Drowsiness Analysis

The system analyzes eye behavior over a sequence of frames to detect signs of drowsiness:

- If eyes are detected closed for more than a set threshold (e.g., 2 seconds), it indicates the driver might be falling asleep.
- This threshold is used to distinguish normal blinking from actual drowsiness.

-

4. Alarm Activation

If drowsiness is detected:

- An alarm or buzzer is triggered to alert the driver.
- The system may continue monitoring and re-activating the alert if drowsiness persists.

5. System Feedback and Optimization

The system can be tested and optimized using real-time recordings or sample test videos by adjusting:

- EAR thresholds
- Frame count sensitivity
- Model tuning (if using custom-trained models)

6.Implementation

Importing Required Modules

```
[ ] import zipfile
    import os

    zip_path = "/content/Drowsiness Detection.v2-augmented-v1.multiclass.zip"
    extract_path = "/content/extracted_data"
```

Extracting a ZIP file

```
[ ] with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_path)
```

Walking Through the Extracted Files

```
[ ] for root, dirs, files in os.walk(extract_path):
        for file in files:
            if 'test' in file.lower():  # You can refine the filter based on file type
                print(os.path.join(root, file))
```

Listing .jpg Images in a Specific Folder

```
[ ] import os

    test_folder = os.path.join(extract_path, "test")  # Adjust if your path is different
    image_files = [os.path.join(test_folder, f) for f in os.listdir(test_folder) if f.endswith('.jpg')]
    print(image_files[:5])  # Show first 5 image paths
```

```
['/content/extracted_data/test/GOPR0492_MP4-394_jpg.rf.8f46f8621777d54593f8b953377894c2.jpg', '/content/extracted_data/test/GOPR0492_MP4-517_jpg.rf.e1d26e0f43076b1ce6569a71
```

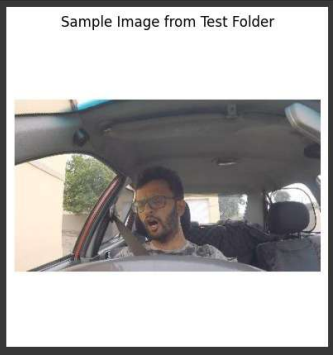
Image Display with Matplotlib

+ Code + Text

```
[ ] import matplotlib.pyplot as plt
    import matplotlib.image as mpimg

    # Display the first image
    img_path = image_files[5]
    img = mpimg.imread(img_path)

    plt.imshow(img)
    plt.axis('off')  # Hide axis
    plt.title("Sample Image from Test Folder")
    plt.show()
```



Preprocessing the Image for a CNN

Preprocessing the Image for a CNN

```
import tensorflow as tf

img_tensor = tf.keras.preprocessing.image.load_img(img_path, target_size=(224, 224))
img_array = tf.keras.preprocessing.image.img_to_array(img_tensor)
img_array = img_array / 255.0  # Normalize if needed
print(img_array.shape)
```

```
(224, 224, 3)
```

```
[ ] import tensorflow as tf
    import os,glob
    import cv2
    from tqdm._tqdm_notebook import tqdm_notebook as tqdm
```

```
/tmp/ipython-input-8-182447216.py:4: TqdmDeprecationWarning: This function will be removed in tqdm==5.0.0
Please use `tqdm.notebook.*` instead of `tqdm._tqdm_notebook.*`
  from tqdm._tqdm_notebook import tqdm_notebook as tqdm
```

Function to Load Dataset

```
[ ] def load_dataset_from_folder(folder_path, image_size=(299, 299)):
        import pandas as pd
        import tensorflow as tf
        from tensorflow.keras.preprocessing.image import load_img, img_to_array
        import os

        csv_path = os.path.join(folder_path, "/content/train.csv")
        df = pd.read_csv(csv_path, sep=",", header=0, names=["filename", "class_0", "class_1"])

        images = []
        labels = []

    # Loop Through Each Row
        for _, row in df.iterrows():
            img_path = os.path.join(folder_path, row['filename'].strip())
            if os.path.exists(img_path):
                image = load_img(img_path, target_size=image_size)
                image = img_to_array(image) / 255.0  # normalize
                label = [int(row['class_0']), int(row['class_1'])]  # convert string to int
                images.append(image)
                labels.append(label)
            else:
                print(f"Image not found: {img_path}")

        return tf.convert_to_tensor(images), tf.convert_to_tensor(labels)
```

```
[ ] # Example usage for training data
    train_folder = os.path.join(extract_path, "train")
    X_train, y_train = load_dataset_from_folder(train_folder)
```

Image Loading & Normalization

Collapse 13 child cells under Image Loading & Normalization (Press <Shift> to also collapse sibling sections)

```
[ ] def load_dataset_from_folder(folder_path, image_size=(299, 299)):
        import pandas as pd
        import tensorflow as tf
        from tensorflow.keras.preprocessing.image import load_img, img_to_array
        import os

        csv_path = os.path.join(folder_path, "/content/test.csv")
        df = pd.read_csv(csv_path, sep=",", header=0, names=["filename", "class_0", "class_1"])

        images = []
        labels = []

        for _, row in df.iterrows():
            img_path = os.path.join(folder_path, row['filename'].strip())
            if os.path.exists(img_path):
                image = load_img(img_path, target_size=image_size)
                image = img_to_array(image) / 255.0  # normalize
                #Label Processing
                label = [int(row['class_0']), int(row['class_1'])]  # convert string to int
                images.append(image)
                labels.append(label)
            else:
                print(f"Image not found: {img_path}")

        return tf.convert_to_tensor(images), tf.convert_to_tensor(labels)
```

```
[ ] # Example usage for training data
    train_folder = os.path.join(extract_path, "test")
    X_test, y_test = load_dataset_from_folder(test_folder)
```

```
[ ] X_test
```

```
array([[[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        ...,
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]]],
```

Show hidden output

```
import os
import pandas as pd
import cv2
import numpy as np
from tqdm import tqdm

def load_dataset_from_folder(folder_path, csv_filename="/content/train.csv", image_size=(224, 224)):
    csv_path = os.path.join(folder_path, csv_filename)
    df = pd.read_csv(csv_path, sep=",", header=0, names=["filename", "class_0", "class_1"])

    X = []
    y = []

    for i in tqdm(range(len(df))):
        row = df.iloc[i]
        img_name = row["filename"]
        img_path = os.path.join(folder_path, img_name)

        if os.path.exists(img_path):
            img = cv2.imread(img_path)
            img = cv2.resize(img, image_size)
            img = img / 255.0  # Normalize to [0, 1]

            X.append(img)

            # Optional: Convert to label string or index
            if int(row["class_0"]) == 1:
                y.append("awake")  # or y.append(0)
            else:
                y.append("drowsy")  # or y.append(1)
        else:
            print(f"Image not found: {img_path}")

    return np.array(X), np.array(y)
```

```
import os
folder_path = "/content/extracted_data/train"
print("CSV exists:", os.path.exists(os.path.join(folder_path, "_classes.csv")))
print("Folder has images:", len(os.listdir(folder_path)) > 0)
```

```
CSV exists: True
Folder has images: True
```

```
X, y = load_dataset_from_folder(folder_path)
print("Loaded images:", len(X))

if len(y) > 0:
    print("Sample label:", y[0])
else:
    print("No label")
```

```
100%|██████████| 1056/1056 [00:01<00:00, 695.09it/s]
Loaded images: 1056
Sample label: drowsy
```

```
print(f"Length of X: {len(X)}")
print(f"Length of y: {len(y)}")
```

```
Length of X: 1056
Length of y: 1056
```

```
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
print("Shape of an image in X_train: ",X_train[0].shape)
print("Shape of an image in X_test: ",X_test[0].shape)
```

```
Shape of an image in X_train:  (224, 224, 3)
Shape of an image in X_test:  (224, 224, 3)
```

```
import numpy as np
print("Unique labels:", np.unique(y))
```

```
Unique labels: ['awake' 'drowsy']
```

```
#import numpy as np
le = preprocessing.LabelEncoder()
y_train = le.fit_transform(y_train)
y_test = le.fit_transform(y_test)
y_train = tf.keras.utils.to_categorical(y_train, num_classes=2)
y_test = tf.keras.utils.to_categorical(y_test, num_classes=2)
y_train = np.array(y_train)
y_test = np.array(y_test)
X_train = np.array(X_train)
X_test = np.array(X_test)
```

```
from ast import Global
from keras.models import Sequential
from keras.layers import Convolution2D, Dropout, Dense, MaxPooling2D, GlobalAveragePooling2D
from keras.layers import BatchNormalization
from keras.layers import MaxPooling2D
from keras.layers import Flatten
from keras import regularizers
def lw(bottom_model, num_classes):
  """creates the top or head of the model that will be placed ontop of the bottom layers"""
  top_model=bottom_model.output
  top_model=GlobalAveragePooling2D()(top_model)
  top_model=Dense(1024,activation='relu')(top_model)
  top_model=Dense(512,activation='relu')(top_model)
  top_model=Dense(128,activation='relu',kernel_regularizer=regularizers.l2(0.01))(top_model)
  top_model=Dense(num_classes,activation='softmax')(top_model)
  return top_model
```

Using MobileNet

- Here's how to replace your current model with MobileN

```
[ ] from keras.applications import MobileNet
    from keras.models import Model
    from keras.layers import GlobalAveragePooling2D, Dense, Dropout

    # Load MobileNet without top classifier layers
    mobilenet = MobileNet(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

    # Freeze all layers
    for layer in mobilenet.layers:
        layer.trainable = False

    # Add custom classification head
    x = GlobalAveragePooling2D()(mobilenet.output)
    x = Dense(128, activation='relu')(x)
    x = Dropout(0.3)(x)
    x = Dense(2, activation='softmax')(x)

    model = Model(inputs=mobilenet.input, outputs=x)

    # Compile the model
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet/mobilenet_1_0_224_tf_no_top.h5
```

```
# Convert to TFLite
import tensorflow as tf
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

# Save
with open("mobilenet_model.tflite", "wb") as f:
    f.write(tflite_model)
```

Show hidden output

Double-click (or enter) to edit

Evaluate Model with AUC, F1-Score, Confusion Matrix

- This gives you a much better understanding than just accuracy.

+ Code + Text

```
[ ] from sklearn.metrics import classification_report, roc_auc_score

    # Predict
    y_pred = model.predict(X_test)
    y_true = np.argmax(y_test, axis=1)
    y_pred_class = np.argmax(y_pred, axis=1)

    # Classification report
    print(classification_report(y_true, y_pred_class))

    # ROC AUC
    print("AUC Score:", roc_auc_score(y_test, y_pred))
```

```
11/11 ━━━━━━━━━━━━━━━━━━━━ 5s 300ms/step
              precision    recall  f1-score   support

           0       0.84      0.89      0.87       200
           1       0.84      0.78      0.81       149

    accuracy                           0.84       349
   macro avg       0.84      0.83      0.84       349
weighted avg       0.84      0.84      0.84       349

AUC Score: 0.920738255033557
```

Use Batch Normalization (if not already using it)

BatchNorm stabilizes and speeds up training.

Add it after dense layers like this:

```
[ ] from keras.layers import BatchNormalization

    x = GlobalAveragePooling2D()(mobilenet.output)
    x = Dense(128, activation='relu')(x)
    x = BatchNormalization()(x)
    x = Dropout(0.3)(x)
    output = Dense(2, activation='softmax')(x)
```

Model Training

```
history=model.fit(X_train,y_train,epochs=50,validation_data=(X_test,y_test), verbose=1, initial_epoch=0)  # call backs early stopping
```

```
Epoch 1/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 17s 420ms/step - accuracy: 0.5726 - loss: 0.8655 - val_accuracy: 0.6160 - val_loss: 0.6467
Epoch 2/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 8s 53ms/step - accuracy: 0.6621 - loss: 0.6388 - val_accuracy: 0.6734 - val_loss: 0.6420
Epoch 3/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - accuracy: 0.7176 - loss: 0.5480 - val_accuracy: 0.7307 - val_loss: 0.5347
Epoch 4/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.7454 - loss: 0.5144 - val_accuracy: 0.7708 - val_loss: 0.4730
Epoch 5/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.7880 - loss: 0.4609 - val_accuracy: 0.7765 - val_loss: 0.4542
Epoch 6/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.7635 - loss: 0.4735 - val_accuracy: 0.8080 - val_loss: 0.4351
Epoch 7/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.7666 - loss: 0.4669 - val_accuracy: 0.8252 - val_loss: 0.4097
Epoch 8/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 57ms/step - accuracy: 0.8305 - loss: 0.3538 - val_accuracy: 0.8138 - val_loss: 0.4219
Epoch 9/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 43ms/step - accuracy: 0.8088 - loss: 0.4257 - val_accuracy: 0.8281 - val_loss: 0.4086
Epoch 10/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.8639 - loss: 0.3376 - val_accuracy: 0.8166 - val_loss: 0.3942
Epoch 11/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 66ms/step - accuracy: 0.8395 - loss: 0.3600 - val_accuracy: 0.8138 - val_loss: 0.4274
Epoch 12/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - accuracy: 0.8597 - loss: 0.3395 - val_accuracy: 0.8252 - val_loss: 0.3827
Epoch 13/50
```

```
Epoch 25/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 57ms/step - accuracy: 0.9093 - loss: 0.2271 - val_accuracy: 0.8424 - val_loss: 0.3487
Epoch 26/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 43ms/step - accuracy: 0.9354 - loss: 0.1775 - val_accuracy: 0.8338 - val_loss: 0.3575
Epoch 27/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 57ms/step - accuracy: 0.8957 - loss: 0.2283 - val_accuracy: 0.8309 - val_loss: 0.3496
Epoch 28/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 3s 63ms/step - accuracy: 0.9319 - loss: 0.1942 - val_accuracy: 0.8453 - val_loss: 0.3449
Epoch 29/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 62ms/step - accuracy: 0.9288 - loss: 0.1668 - val_accuracy: 0.8367 - val_loss: 0.3994
Epoch 30/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.9081 - loss: 0.2334 - val_accuracy: 0.8252 - val_loss: 0.4042
Epoch 31/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.9281 - loss: 0.1937 - val_accuracy: 0.8252 - val_loss: 0.4621
Epoch 32/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.9198 - loss: 0.1991 - val_accuracy: 0.8309 - val_loss: 0.3532
Epoch 33/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 57ms/step - accuracy: 0.9510 - loss: 0.1526 - val_accuracy: 0.8109 - val_loss: 0.4462
Epoch 34/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 57ms/step - accuracy: 0.9343 - loss: 0.1681 - val_accuracy: 0.8080 - val_loss: 0.5112
Epoch 35/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.9056 - loss: 0.2136 - val_accuracy: 0.8424 - val_loss: 0.3380
Epoch 36/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.9612 - loss: 0.1339 - val_accuracy: 0.8424 - val_loss: 0.3513
Epoch 37/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.9572 - loss: 0.1285 - val_accuracy: 0.8567 - val_loss: 0.3539
Epoch 38/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 58ms/step - accuracy: 0.9633 - loss: 0.1352 - val_accuracy: 0.8309 - val_loss: 0.3701
Epoch 39/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.9563 - loss: 0.1228 - val_accuracy: 0.8395 - val_loss: 0.3911
Epoch 40/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.9643 - loss: 0.1102 - val_accuracy: 0.8424 - val_loss: 0.3689
Epoch 41/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.9797 - loss: 0.0989 - val_accuracy: 0.8252 - val_loss: 0.3825
Epoch 42/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.9825 - loss: 0.0826 - val_accuracy: 0.8338 - val_loss: 0.3891
Epoch 43/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.9667 - loss: 0.0933 - val_accuracy: 0.8367 - val_loss: 0.3932
Epoch 44/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 57ms/step - accuracy: 0.9820 - loss: 0.0818 - val_accuracy: 0.8481 - val_loss: 0.3874
Epoch 45/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.9773 - loss: 0.0820 - val_accuracy: 0.8252 - val_loss: 0.3969
Epoch 46/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 57ms/step - accuracy: 0.9577 - loss: 0.1257 - val_accuracy: 0.8481 - val_loss: 0.3987
Epoch 47/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 3s 60ms/step - accuracy: 0.9564 - loss: 0.1187 - val_accuracy: 0.8481 - val_loss: 0.3771
Epoch 48/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 43ms/step - accuracy: 0.9739 - loss: 0.0861 - val_accuracy: 0.8395 - val_loss: 0.4147
Epoch 49/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.9774 - loss: 0.0789 - val_accuracy: 0.8481 - val_loss: 0.4064
Epoch 50/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.9794 - loss: 0.0665 - val_accuracy: 0.8424 - val_loss: 0.4148
```

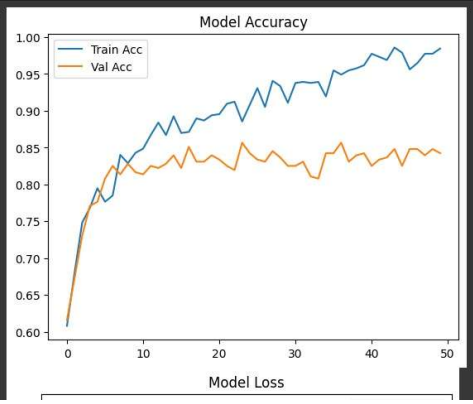
Visualize Model Performance

- See if training and validation accuracy are stable:

```
import matplotlib.pyplot as plt

# Accuracy
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title("Model Accuracy")
plt.legend()
plt.show()

# Loss
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title("Model Loss")
plt.legend()
plt.show()
```



Model Loss


Model Loss
Train Loss
Val Loss
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
10
20
30
40
50

Final Thought

- If MobileNet works better, stick with it — there's no need to use heavier models unless accuracy plateaus or you need to handle more complex image patterns.

Let me know if you want:

To try EfficientNet

Help deploying your model

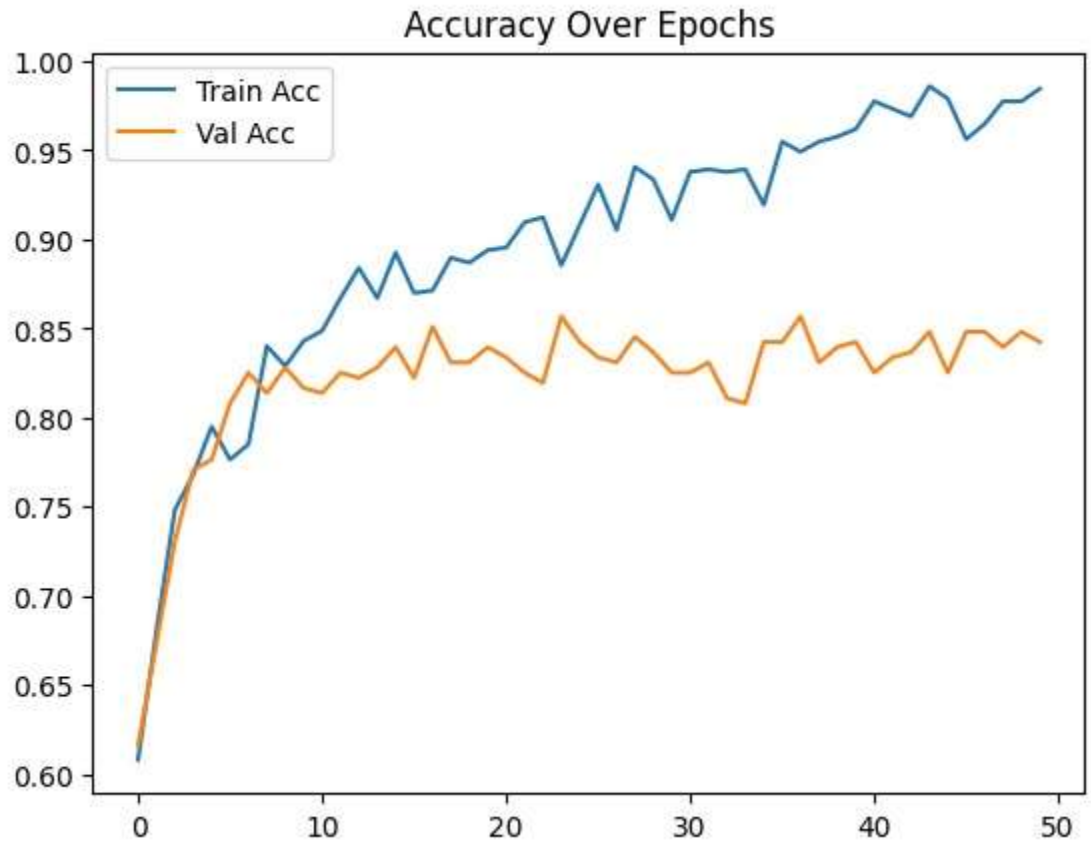
Code to convert MobileNet to TensorFlow Lite or ONNX for mobile/edge

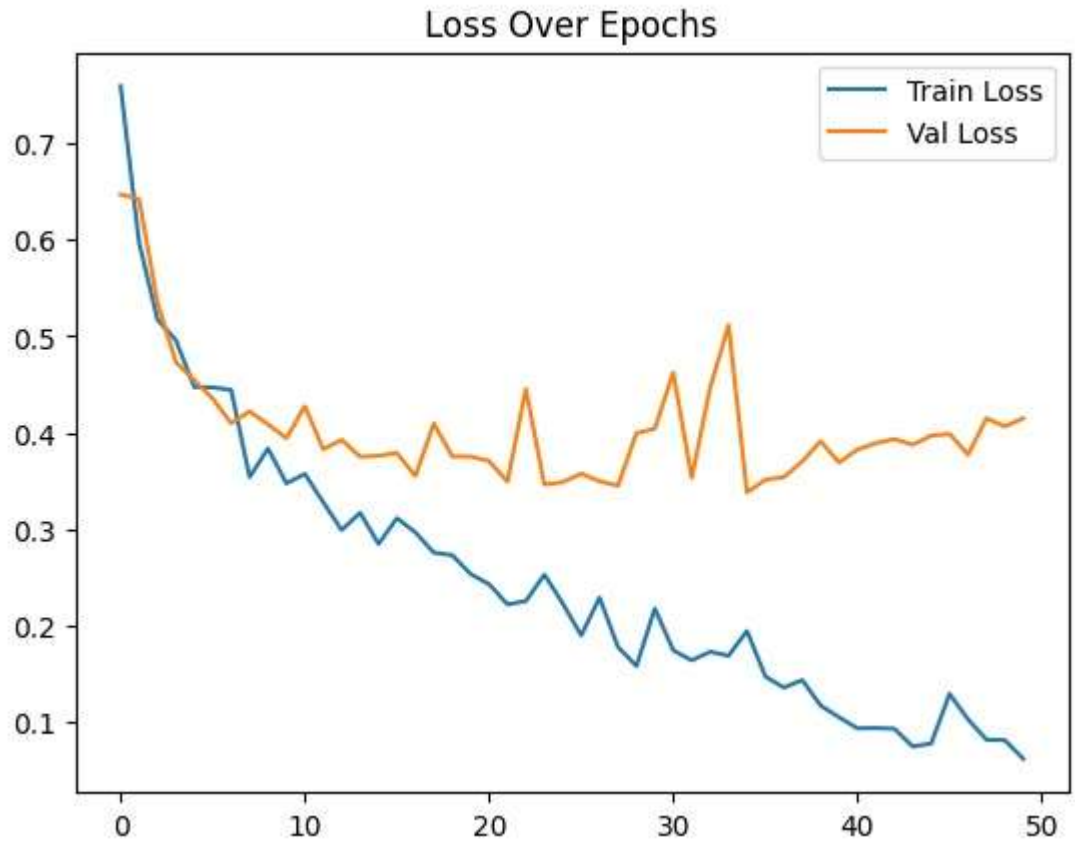
Great job picking the right tool for your task!

```
[ ]  import matplotlib.pyplot as plt

    # Accuracy plot
    plt.plot(history.history['accuracy'], label='Train Acc')
    plt.plot(history.history['val_accuracy'], label='Val Acc')
    plt.title("Accuracy Over Epochs")
    plt.legend()
    plt.show()

    # Loss plot
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Val Loss')
    plt.title("Loss Over Epochs")
    plt.legend()
    plt.show()
```




Loss Over Epochs
Train Loss
Val Loss
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
10
20
30
40
50

---

6. Advantages of Machine Learning in Drowsiness Car Driving Detection

---

- ☑ **Real-Time Monitoring and Alerts**

Machine learning enables real-time monitoring of the driver's face and eyes using webcam input. The system can immediately detect signs of drowsiness and trigger an alert without delay.

- 😴 **Improved Road Safety**

By detecting fatigue early, the system helps in preventing road accidents. This is especially useful for long-distance drivers or people driving late at night.

- 📊 **Accurate Detection Based on Facial Patterns**

Using computer vision and deep learning, the model accurately identifies closed eyes, blinking patterns, and other facial indicators of sleepiness.

- ⚡ **Fast and Efficient**

Once the model is integrated and running, it works automatically and continuously in the background with minimal resources and instant output.

 Handles Complex Facial Inputs
The system processes live video input with many variables like lighting, head movement, and face angles, which machine learning handles efficiently.
 Automated Alerts Without Human Intervention
The model triggers alarms when drowsiness is detected, reducing dependence on others to monitor the driver.
 Useful for Transport and Automotive Industry
This project can be integrated into vehicles or fleet systems to monitor drivers and reduce liability due to fatigue-related accidents.
 Scalable and Customizable
The model can be retrained or improved with more real-world data and adapted for use in commercial vehicles, smart helmets, or driver assistance systems.

Conclusion

Through this project, I learned how machine learning and computer vision can be used to solve real-life problems such as preventing road accidents caused by drowsy driving. By using a webcam, deep learning models like **MobileNet**, and tools like **OpenCV**, I built a system that detects when a driver is feeling sleepy and alerts them through an alarm.

This hands-on project gave me valuable experience in areas like face detection, eye aspect ratio (EAR) calculation, and real-time video processing. Initially, I faced challenges such as tuning detection thresholds and managing real-time input, but step by step, I understood how to fine-tune the model and improve accuracy.

I also learned how small visual cues like blinking speed and eye closure can be used to predict human behavior using AI. Working with Python and machine learning libraries helped me strengthen both my coding and problem-solving skills.

What I liked most about this project is that it's not just theoretical—it has real-world impact. It showed me how technology can save lives and make driving safer. I now feel more confident in doing more advanced machine learning projects and contributing to smart and safe systems in the future.

In conclusion, this project improved my technical knowledge, increased my confidence in using AI tools, and motivated me to continue exploring the field of intelligent safety systems. I'm proud of what I've achieved and excited to learn more in this journey.